

Contents

1	Redes neuronales para bioinformática	1
1.1	De modelos explícitos a representaciones aprendidas	1
1.2	El Perceptrón y la no linealidad	1
1.2.1	De la neurona biológica a la unidad de cómputo	1
1.2.2	La necesidad matemática de la no linealidad	2
1.3	Entrenamiento, pérdida y retropropagación	3
1.3.1	Función de pérdida de Entropía Cruzada Binaria	3
1.3.2	Optimización por Descenso de Gradiente	3
1.3.3	El algoritmo de Retropropagación (<i>Backpropagation</i>)	3
1.4	Caso de estudio resuelto: predicción de enhancers	4
1.4.1	Paso hacia adelante (<i>Forward Pass</i>)	4
1.4.2	Evaluación de la función de pérdida	5
1.4.3	Paso hacia atrás (<i>Backward Pass</i>) y actualización de pesos	5
1.5	Sobreajuste y alta dimensionalidad en ciencias ómicas	7
1.6	Responsabilidad, ética y límites del aprendizaje automático	7
1.7	La operación peligrosa: fugas de datos y saturación	8
1.8	Actividad de depuración: errores en la retropropagación	8
1.9	Síntesis operativa y tablas de diagnóstico	9
1.9.1	Tabla de diagnóstico de anomalías en entrenamiento	9
1.9.2	Tabla de síntesis con preguntas de control	9
1.10	Glosario conceptual	9
1.11	Cierre del libro: la caja de herramientas computacionales	10
1.12	Fuentes y reproducibilidad	10
	Anexo práctico · Entrenar, propagar gradientes y auditar un modelo	11

Chapter 1

Redes neuronales para bioinformática

1.1 De modelos explícitos a representaciones aprendidas

A lo largo de los capítulos anteriores hemos construido modelos con reglas, matrices o distribuciones explícitamente diseñadas por el analista: Esencial

- En **Naïve-Bayes** (TC-CH08), asumimos independencia condicional y estimamos frecuencias directas de nucleótidos.
- En **Cadenas y Modelos Ocultos de Markov** (TC-CH08), fijamos matrices de transición estocásticas para capturar dependencias de orden 1.
- En **Alineamiento de secuencias** (TC-CH09), definimos a mano matrices de sustitución (PAM, BLOSUM) y penalizaciones afines de hueco.

Sin embargo, cuando nos enfrentamos a problemas biomédicos complejos —como integrar simultáneamente múltiples pistas epigenéticas, perfiles de expresión génica masiva o predecir la actividad de regiones reguladoras distales (*enhancers*)—, las interacciones biológicas no lineales y combinatorias resultan imposibles de codificar mediante reglas estáticas artesanales.

Este capítulo culmina el libro introduciendo las **Redes Neuronales Artificiales**: arquitecturas parametrizadas capaces de **aprender representaciones jerárquicas y fronteras de decisión complejas directamente a partir de los datos**.

Las redes neuronales representan la transición desde modelos probabilísticos artesanales y reglas de puntuación fijas hacia arquitecturas parametrizadas que aprenden representaciones jerárquicas y fronteras de decisión no lineales directamente a partir de datos biológicos.

La ruta **esencial** cubre la estructura del perceptrón, la necesidad matemática de funciones de activación no lineales, la función de pérdida de entropía cruzada, la optimización por descenso de gradiente y retropropagación (*backpropagation*), el caso de estudio resuelto de predicción de *enhancers* epigenéticos, y el marco de responsabilidad y auditoría ética (CON-ETHICS / TC-C7); 120–140 minutos. La ruta de **estudio** profundiza en el análisis del sobreajuste en regímenes de alta dimensionalidad ($p \gg n$), el fenómeno de la saturación de gradientes, la actividad de depuración y el laboratorio práctico; 60–80 minutos adicionales. Esencial

1.2 El Perceptrón y la no linealidad

1.2.1 De la neurona biológica a la unidad de cómputo

El **Perceptrón** (Rosenblatt, 1958) es la unidad atómica del aprendizaje profundo, inspirado en el funcionamiento electroquímico de las neuronas biológicas:

1. **Dendritas (Entradas):** Reciben el vector de características biológicas $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ (p. ej., niveles de metilación o accesibilidad cromatínica).
2. **Soma (Combinación lineal):** Realiza una suma ponderada de las entradas modulada por los pesos sinápticos $\mathbf{w}$ y un sesgo basal (*bias*) b :

$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

3. **Axón (Activación y Salida):** Transforma el potencial lineal z mediante una **función de activación no lineal** $f(z)$ para emitir la activación final $a = f(z)$.

El perceptrón artificial modela una neurona biológica calculando una combinación afín $z = \sum w_i x_i + b$ y aplicando una función de activación $a = f(z)$ para generar la salida.

1.2.2 La necesidad matemática de la no linealidad

Si utilizáramos funciones de activación lineales ($f(z) = c \cdot z$), una red neuronal con múltiples capas ocultas colapsaría matemáticamente a una única transformación afín equivalente:

$$\mathbf{y} = \mathbf{W}_2(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2 \mathbf{W}_1) \mathbf{x} + (\mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{\text{eq}} \mathbf{x} + \mathbf{b}_{\text{eq}}$$

Una red puramente lineal es incapaz de resolver problemas no linealmente separables, como la compuerta lógica XOR o la interacción cooperativa entre dos factores de transcripción que solo activan un gen si ambos están presentes en concentraciones intermedias.

Las funciones de activación no lineales (Sigmoide, ReLU, Tanh) son matemáticamente indispensables porque una composición de capas puramente lineales colapsa a una única transformación lineal, incapaz de resolver problemas no linealmente separables como el XOR biológico.

Funciones de activación canónicas:

- **Sigmoide (σ):** $\sigma(z) = \frac{1}{1+e^{-z}}$, con codominio en $(0, 1)$. Es idónea para capas de salida en clasificación binaria porque su valor se interpreta como una probabilidad $P(y = 1 \mid \mathbf{x})$. Su derivada analítica es $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Unidad Lineal Rectificada (ReLU):** $\text{ReLU}(z) = \max(0, z)$. Es el estándar en capas ocultas profundas porque su derivada es constante (1) para $z > 0$, evitando la saturación.
- **Tangente hiperbólica (tanh):** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, centrada en cero con codominio en $(-1, 1)$.

El peligro de la saturación del gradiente en la sigmoide: Cuando el valor absoluto de la preactivación es elevado ($|z| \gg 0$), la función sigmoide se vuelve prácticamente plana ($\sigma(z) \approx 0$ o $\sigma(z) \approx 1$). En ese régimen, su derivada tiende a cero:

$$\lim_{|z| \rightarrow \infty} \sigma'(z) = \lim_{|z| \rightarrow \infty} \sigma(z)(1 - \sigma(z)) = 0$$

Esto provoca el fenómeno del **desvanecimiento del gradiente** (*vanishing gradient*), congelando el aprendizaje de los pesos en capas anteriores.

Cuando las entradas a una función sigmoide tienen gran magnitud absoluta ($|z| \gg 0$), la derivada $\sigma'(z)$ tiende a cero, provocando el desvanecimiento del gradiente y deteniendo el aprendizaje de los pesos en capas precedentes. Se reproduce con `verification/chapter_results.sh saturation_control 10.0`.

1.3 Entrenamiento, pérdida y retropropagación

El entrenamiento de una red neuronal consiste en encontrar el conjunto de pesos $\mathbf{W}$ y sesgos $\mathbf{b}$ que minimicen la discrepancia entre las predicciones del modelo $\hat{y}$ y las etiquetas biológicas reales y .

1.3.1 Función de pérdida de Entropía Cruzada Binaria

Para tareas de clasificación binaria supervisada con $y \in \{0, 1\}$ y $\hat{y} = P(y = 1 \mid \mathbf{x}) \in (0, 1)$, la función de pérdida canónica es la **Entropía Cruzada Binaria** (*Binary Cross-Entropy*, BCE), fundamentada como el negativo de la log-verosimilitud de una distribución de Bernoulli:

$$P(y \mid \mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y} \implies -\ln P(y \mid \mathbf{x}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

$$\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

Esta formulación conecta directamente con la divergencia KL y la entropía cruzada formalizadas en TC-CH07.

La función de pérdida de Entropía Cruzada Binaria mide la divergencia entre la etiqueta real $y \in \{0, 1\}$ y la probabilidad predicha $\hat{y}$ mediante $\mathcal{L}(y, \hat{y}) = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$.

1.3.2 Optimización por Descenso de Gradiente

El algoritmo de **Descenso de Gradiente** ajusta iterativamente cada peso en la dirección opuesta al gradiente de la función de pérdida:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}}$$

donde $\eta > 0$ es la **tasa de aprendizaje** (*learning rate*).

El descenso de gradiente actualiza iterativamente los parámetros en la dirección opuesta al gradiente de la pérdida según $w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}$, donde η es la tasa de aprendizaje.

Impacto de la tasa de aprendizaje η :

- Si η es excesivamente grande ($\eta = 10.0$), el algoritmo da pasos desproporcionados, sobrepasa el mínimo y diverge numéricamente.
- Si η es excesivamente pequeña ($\eta = 10^{-4}$), el entrenamiento se estanca en mesetas de gradiente nulo.

Una tasa de aprendizaje excesivamente grande provoca divergencia numérica u oscilaciones destructivas sobre la superficie de pérdida, mientras que una tasa excesivamente pequeña estanca la convergencia; un ajuste riguroso de hiperparámetros es indispensable para la estabilidad del entrenamiento. Se reproduce con `verification/chapter_results.sh learning_rate_perturbation 0.1`.

1.3.3 El algoritmo de Retropropagación (*Backpropagation*)

El algoritmo de **Retropropagación** (Rumelhart, Hinton & Williams, 1986) calcula de forma eficiente las derivadas parciales $\frac{\partial \mathcal{L}}{\partial w_{ij}}$ aplicando la **regla de la cadena** del cálculo multivariable en sentido inverso, desde la capa de salida hacia la capa de entrada.

La retropropagación calcula las derivadas parciales exactas de la función de pérdida respecto a cada peso y sesgo aplicando la regla de la cadena del cálculo multivariable de forma recursiva desde la capa de salida hacia las capas ocultas.

1.4 Caso de estudio resuelto: predicción de enhancers

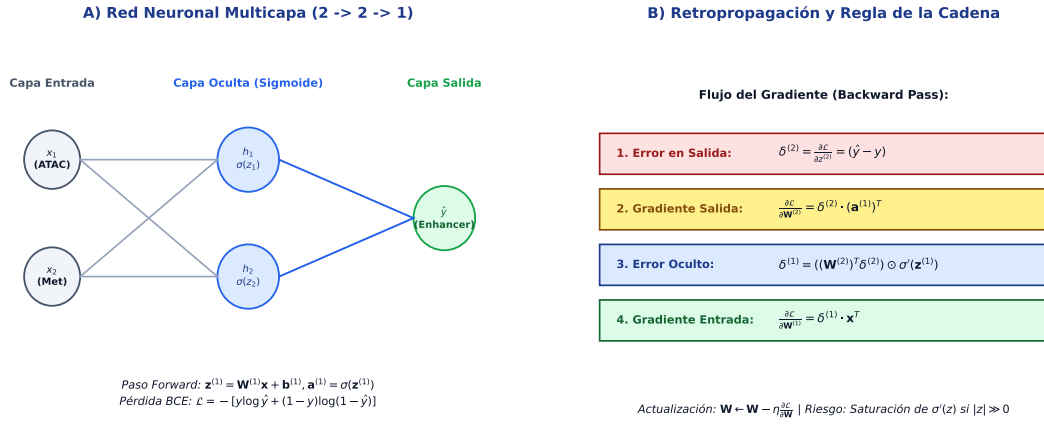


Figure 1.1: A) Arquitectura de Red Neuronal Multicapa ($2 \rightarrow 2 \rightarrow 1$) para clasificación epigenómica de enhancers activos. B) Flujo de retropropagación (*backpropagation*) y cálculo formal de gradientes mediante la regla de la cadena. Figura original adaptada de las presentaciones docentes.

Consideremos una red neuronal feedforward con arquitectura $2 \rightarrow 2 \rightarrow 1$ diseñada para predecir si una región genómica es un **enhancer activo** ($y = 1$) a partir de dos biomarcadores epigenéticos continuos:

- $x_1 = 1.0$: Señal normalizada de enriquecimiento de la marca de histona activa H3K27ac.
- $x_2 = 0.5$: Accesibilidad cromatínica cuantificada por ATAC-seq.

Parámetros iniciales de la red:

- **Capa oculta (2 neuronas con activación sigmoide):**
 - Neurona h_1 : Pesos $w_{11}^{(1)} = 0.5$, $w_{12}^{(1)} = -0.3$; sesgo $b_1^{(1)} = 0.1$.
 - Neurona h_2 : Pesos $w_{21}^{(1)} = -0.2$, $w_{22}^{(1)} = 0.8$; sesgo $b_2^{(1)} = -0.2$.
- **Capa de salida (1 neurona con activación sigmoide):** Pesos $w_1^{(2)} = 0.7$, $w_2^{(2)} = -0.4$; sesgo $b^{(2)} = 0.2$.

1.4.1 Paso hacia adelante (*Forward Pass*)

1. Activaciones de la capa oculta:

- Preactivación de h_1 :

$$z_{h1} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} = (0.5)(1.0) + (-0.3)(0.5) + 0.1 = 0.5 - 0.15 + 0.1 = \mathbf{0.45}$$

Activación de h_1 :

$$a_{h1} = \sigma(0.45) = \frac{1}{1 + e^{-0.45}} \approx \mathbf{0.610639}$$

- Preactivación de h_2 :

$$z_{h2} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)} = (-0.2)(1.0) + (0.8)(0.5) - 0.2 = -0.2 + 0.4 - 0.2 = \mathbf{0.00}$$

Activación de h_2 :

$$a_{h2} = \sigma(0.00) = \frac{1}{1 + e^0} = \mathbf{0.500000}$$

2. Activación de la capa de salida:

- Preactivación de salida:

$$z_{\text{out}} = w_1^{(2)} a_{h1} + w_2^{(2)} a_{h2} + b^{(2)} = (0.7)(0.610639) + (-0.4)(0.500000) + 0.2 \approx \mathbf{0.427447}$$

- Probabilidad predicha final ($\hat{y}$):

$$\hat{y} = \sigma(0.427447) = \frac{1}{1 + e^{-0.427447}} \approx \mathbf{0.605264}$$

En la red 2-2-1 de predicción de enhancers con entradas epigenéticas $x_1 = 1.0$ (H3K27ac) y $x_2 = 0.5$ (ATAC-seq), el paso forward calcula las preactivaciones ocultas $z_{h1} = 0.45$ ($a_{h1} = 0.610639$) y $z_{h2} = 0.00$ ($a_{h2} = 0.500000$), produciendo una preactivación de salida $z_{\text{out}} = 0.427447$ y una predicción $\hat{y} = 0.605264$. Se reproduce con `verification/chapter_results.sh forward_enhancer_pass 1.0 0.5`.

1.4.2 Evaluación de la función de pérdida

Dado que la muestra observada corresponde experimentalmente a un enhancer activo ($y = 1$):

$$\mathcal{L}(1, 0.605264) = -\ln(0.605264) \approx \mathbf{0.502091}$$

Para la etiqueta real $y = 1$ (enhancer activo) y la predicción $\hat{y} = 0.605264$, la pérdida de Entropía Cruzada Binaria evalúa a $-\ln(0.605264) = 0.502091$. Se reproduce con `verification/chapter_results.sh bce_loss 1.0 0.605264`.

1.4.3 Paso hacia atrás (*Backward Pass*) y actualización de pesos

Para la combinación de pérdida de entropía cruzada con salida sigmoide, el error en el nodo de salida se simplifica elegantemente a:

$$\delta_{\text{out}} = \frac{\partial \mathcal{L}}{\partial z_{\text{out}}} = \hat{y} - y = 0.605264 - 1.0 = \mathbf{-0.394736}$$

Gradientes de los pesos de la capa de salida:

- $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = \delta_{\text{out}} \cdot a_{h1} = (-0.394736)(0.610639) \approx \mathbf{-0.241041}$
- $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = \delta_{\text{out}} \cdot a_{h2} = (-0.394736)(0.500000) \approx \mathbf{-0.197368}$

Actualización de pesos con tasa de aprendizaje $\eta = 0.1$:

- $w_1^{(2)} \leftarrow 0.7 - (0.1)(-0.241041) = 0.7 + 0.024104 = \mathbf{0.724104}$
- $w_2^{(2)} \leftarrow -0.4 - (0.1)(-0.197368) = -0.4 + 0.019737 = \mathbf{-0.380263}$

El peso sináptico asociado a la marca activa H3K27ac ($w_1^{(2)}$) se incrementa de forma coherente para aumentar la probabilidad en la siguiente iteración.

En el paso backward para $y = 1$, el error de salida δ_{out} es -0.394736 , produciendo los gradientes $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}} = -0.241041$ y $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}} = -0.197368$; actualizando con $\eta = 0.1$ se obtiene el nuevo peso $w_1^{(2)} = 0.724104$. Se reproduce con `verification/chapter_results.sh backward_enhancer_pass 1.0 0.5 1.0 0.1`.

Listing 1.1: Algoritmo formal en pseudocódigo para el pase Forward y Backward en una Red Neuronal $2 \rightarrow 2 \rightarrow 1$.

```

ALGORITMO: Red_Neuronal_Forward_Backward_2x2x1
ENTRADA:
- x = [x1, x2]: Vector de características de entrada (ej. ATAC-seq y metilacion)
- y: Etiqueta real binaria en {0, 1}
- eta: Tasa de aprendizaje (learning rate, ej. 0.1)
- W1 (2x2), b1 (2x1), W2 (1x2), b2 (1x1): Parametros actuales de la red
SALIDA:
- y_hat: Probabilidad predicha en [0, 1]
- Perdida_BCE: Perdida de entropia cruzada binaria
- W1_nuevo, b1_nuevo, W2_nuevo, b2_nuevo: Parametros actualizados por descenso de
  gradiente
COMPLEJIDAD: O(1) tiempo por iteracion, O(1) espacio

1. PASO HACIA ADELANTE (FORWARD PASS):
z1 <- W1 * x + b1 # Preactivacion capa oculta (2x1)
a1 <- [sigma(z1[0]), sigma(z1[1])] # Activacion sigmoide sigma(z) = 1 / (1 + exp(-
  z))

z2 <- W2 * a1 + b2 # Preactivacion capa de salida (1x1)
y_hat <- sigma(z2) # Prediccion final

Perdida_BCE <- - (y * log(y_hat) + (1 - y) * log(1 - y_hat))

2. PASO HACIA ATRAS (BACKWARD PASS):
delta_salida <- y_hat - y # Error en salida (gradiente BCE + Sigmoide)

# Gradientes capa 2 (Salida)
grad_W2 <- delta_salida * a1^T
grad_b2 <- delta_salida

# Propagacion del error hacia la capa oculta 1
sigma_prime_z1 <- [a1[0] * (1 - a1[0]), a1[1] * (1 - a1[1])]
delta_oculto <- (W2^T * delta_salida) (producto elemento a elemento)
sigma_prime_z1

# Gradientes capa 1 (Oculta)
grad_W1 <- delta_oculto * x^T
grad_b1 <- delta_oculto

3. ACTUALIZACION DE PARAMETROS (DESCENSO DE GRADIENTE):
W2_nuevo <- W2 - eta * grad_W2

```



```

b2_nuevo <- b2 - eta * grad_b2
W1_nuevo <- W1 - eta * grad_W1
b1_nuevo <- b1 - eta * grad_b1

```

4. RETORNO:

```
Retornar (y_hat, Perdida_BCE, W1_nuevo, b1_nuevo, W2_nuevo, b2_nuevo)
```

1.5 Sobreajuste y alta dimensionalidad en ciencias ómicas

En biomedicina computacional es ubicuo el régimen de **alta dimensionalidad** ($p \gg n$): disponemos de la expresión de $p = 20\,000$ genes o millones de variantes para una cohorte clínica de solo $n = 100$ pacientes.

En este escenario, una red neuronal profunda con miles de parámetros libres puede memorizar con facilidad el ruido muestral de entrenamiento, alcanzando un 100% de exactitud aparente pero fallando catastróficamente al predecir pacientes de un nuevo hospital (**sobreajuste** / *overfitting*).

Técnicas esenciales de regularización:

- **Regularización L_2 (*Weight Decay*):** Penaliza pesos grandes sumando $\frac{1}{2}\lambda\|\mathbf{w}\|^2$ a la pérdida.
- **Desconexión aleatoria (*Dropout*):** Desactiva aleatoriamente una fracción de neuronas (p. ej., 20–50%) en cada paso de entrenamiento para evitar coadaptaciones espurias.
- **Parada temprana (*Early Stopping*):** Monitoriza la pérdida en un conjunto de validación independiente y detiene el entrenamiento antes de que comience el sobreajuste.

En datos ómicos, la alta dimensionalidad ($p \gg n$, miles de variables de expresión génica frente a decenas de pacientes) crea un elevado riesgo de sobreajuste, donde el modelo memoriza el ruido de entrenamiento sin generalizar a nuevas cohortes clínicas.

1.6 Responsabilidad, ética y límites del aprendizaje automático

El resultado de aprendizaje oficial **TC-C7** y la propiedad conceptual **CON-ETHICS** exigen que el futuro biotecnólogo y bioinformático audite críticamente las limitaciones éticas y metodológicas de los modelos de caja negra.

1. Correlación estadística frente a causalidad biológica: Una red neuronal detecta asociaciones estadísticas en la distribución de entrenamiento; **no demuestra causalidad biológica ni función mecanística**. Un biomarcador con alto peso predictivo puede ser un subproducto secundario de la enfermedad o un artefacto técnico (*efecto de lote* o *batch effect* del secuenciador).

2. Sesgo poblacional y demográfico: Si un modelo de riesgo poligénico o de respuesta a fármacos se entrena mayoritariamente sobre genomas de ascendencia europea, su rendimiento se degrada drásticamente al aplicarse sobre poblaciones históricamente subrepresentadas, pudiendo inducir disparidades diagnósticas y daños clínicos.

Las predicciones de aprendizaje automático en bioinformática reflejan correlaciones estadísticas en los datos de entrenamiento y no establecen causalidad biológica ni sustituyen la validación experimental o el juicio clínico; los modelos deben auditarse frente a sesgos demográficos y aprendizaje de atajos espurios.

3. Protocolo de validación clínica en 3 niveles y prevención de fugas: Toda aplicación biomédica de IA debe estructurarse en tres niveles metodológicos independientes:

1. **Entrenamiento y validación interna:** Ajuste de preprocesamiento, pesos e hiperparámetros mediante validación cruzada agrupada por paciente (*GroupKFold*). En un estudio con 1000 muestras de 100 pacientes, dividir filas al azar filtra la identidad del paciente entre entrenamiento y test (*data leakage*), haciendo que la red memorice al individuo en vez de la enfermedad.
2. **Conjunto de prueba congelado (*Test Set*):** Evaluación final única sobre datos retenidos desde el inicio, sin reajustar ningún hiperparámetro.
3. **Validación externa independiente:** Evaluación en una cohorte procedente de otro hospital o tecnología para comprobar la generalización real (*out-of-distribution*).

Métricas y supervisión: Reportar obligatoriamente sensibilidad, especificidad y AUC-ROC junto a intervalos de confianza, manteniendo la supervisión humana como principio innegociable.

La auditoría de riesgos de un modelo exige validación cruzada estratificada en cohortes independientes sin fugas de datos y evaluación mediante métricas clínicas completas (sensibilidad, especificidad, AUC-ROC) en lugar de la exactitud global agregada.

1.7 La operación peligrosa: fugas de datos y saturación

Los fallos más graves en proyectos bioinformáticos de aprendizaje automático son:

1. **Fuga de datos (*Data Leakage*):** Normalizar o seleccionar genes sobre la totalidad del dataset antes de dividir en entrenamiento y test, generando métricas optimistas falsas.
2. **Tasas de aprendizaje inadecuadas:** Divergencia silenciosa a valores NaN o estancamiento por saturación sigmoidea.

Protocolo preventivo en 3 pasos:

1. **Aislamiento del conjunto de prueba:** Congelar el test set antes de cualquier preprocesamiento, imputación o escalado.
2. **Aserción de gradientes finitos:** Comprobar tras cada época que ningún peso contenga valores infinitos o NaN.
3. **Diagnóstico de interpretabilidad:** Emplear mapas de saliencia o valores SHAP para formular hipótesis auditables sobre las variables relevantes, contrastándolas siempre con modelos de referencia (*baselines* lineales) y sin asumirlas como demostración causal o prueba de ausencia de atajos.

1.8 Actividad de depuración: errores en la retropropagación

Analice el siguiente fragmento con tres errores conceptuales en la actualización de una red:

Listing 1.2: Fragmento con tres errores en la actualización de pesos.

```
backward_step(x, y_true, y_hat, a_hidden, W_out, eta):
    # Fallo 1: el error de salida se calcula con signo invertido
    delta_out <- y_true - y_hat
```

```
# Fallo 2: se actualiza el peso sumando el gradiente en vez de restarlo (ascenso de perdida)
W_out[1] <- W_out[1] + (eta * delta_out * a_hidden[1])

# Fallo 3: se reutiliza W_out ya modificado para propagar hacia la capa previa
grad_hidden <- delta_out * W_out[1]
devolver W_out
```

1.9 Síntesis operativa y tablas de diagnóstico

1.9.1 Tabla de diagnóstico de anomalías en entrenamiento

Síntoma observado	Causa probable	Comprobación y corrección
La pérdida devuelve NaN en las primeras épocas	Tasa de aprendizaje η excesivamente grande	Reducir η (p. ej., de 1.0 a 10^{-3})
La pérdida no desciende y los gradientes son cero	Saturación de activaciones sigmoideas ($ z \gg 0$)	Inicializar pesos con Xavier/He o usar ReLU
Exactitud 99% en train pero 50% en test clínico	Sobreajuste severo por régimen $p \gg n$	Aplicar Dropout, regularización L_2 y validación externa
El modelo predice la misma clase para todas las muestras	Desbalance extremo de clases o sesgo colapsado	Balancear pesos de clase en la pérdida o submuestrear

1.9.2 Tabla de síntesis con preguntas de control

Fase del proyecto	Técnica recomendada	Pregunta de control
Diseño de arquitectura	Capas densas con ReLU / salida Sigmoide	¿Se justifica la no linealidad frente a modelos lineales?
Optimización	Descenso de gradiente con BCE	¿Se verifica la estabilidad numérica de la pérdida?
Evaluación ética	Validación cruzada en cohortes externas	¿Se han descartado sesgos demográficos y fugas de datos?

1.10 Glosario conceptual

- **Perceptrón:** unidad de cómputo lineal con activación no lineal.
- **Sigmoide:** función $1/(1+e^{-z})$ con codominio en $(0, 1)$.
- **ReLU:** rectificador lineal $\max(0, z)$ para redes profundas.
- **Entropía cruzada:** función de pérdida canónica para clasificación probabilística.
- **Backpropagation:** cálculo de gradientes mediante la regla de la cadena.
- **Learning rate (η):** tamaño de paso en el descenso de gradiente.
- **Vanishing gradient:** desvanecimiento de derivadas en regiones saturadas.
- **Overfitting:** memorización de ruido muestral en lugar de patrones reales.
- **Dropout:** regularización por desactivación estocástica de neuronas.
- **Data leakage:** filtración de información del test set al entrenamiento.

1.11 Cierre del libro: la caja de herramientas computacionales

Este capítulo final integra el conjunto de herramientas computacionales desarrollado a lo largo de todo el libro: desde la automatización reproducible en terminal Unix (TC-CH01–TC-CH02), el análisis asintótico de eficiencia y estructuras de datos (TC-CH03–TC-CH06), la cuantificación de incertidumbre por teoría de la información (TC-CH07), y los modelos probabilísticos y de alineamiento (TC-CH08–TC-CH09), hasta el aprendizaje de representaciones mediante redes neuronales.

El estudiante dispone ahora de un criterio técnico riguroso: sabe cuándo basta un script de Bash con expresiones regulares, cuándo formular un modelo probabilístico transparente con cadenas de Markov o programación dinámica, y cuándo recurrir al aprendizaje automático supervisado, auditando siempre con responsabilidad ética y científica los límites de cada herramienta.

1.12 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas docentes (20251202_bioinfo_6.pptx, Álvaro Serrano Navarro), en los artículos fundacionales de Rosenblatt (1958) y Rumelhart et al. (1986), y en el tratado de Goodfellow et al. (2016). Todos los cálculos matemáticos y actualizaciones de gradiente se reproducen de forma determinista mediante `verification/chapter_results.sh`.

Anexo práctico · Entrenar, propagar gradientes y auditar un modelo

Pregunta, producto y separación de materiales

¿Puede calcular a mano el paso forward y backward de una red neuronal para predicción de enhancers epigenéticos, evaluar el impacto de perturbar la tasa de aprendizaje, verificar los regímenes de saturación del gradiente y formular una auditoría ética rigurosa frente a sesgos biomédicos? Este laboratorio responde que sí, de forma totalmente reproducible.

El producto individual contiene: el cálculo manual del paso forward y la pérdida de entropía cruzada contrastados con el script, el paso backward con actualización de pesos, la perturbación de la tasa de aprendizaje, los controles de caso límite en saturación y ceros, y la auditoría ética del modelo, todo en `notas/respuestas.tsv`.

El anexo es autónomo e independiente: no requiere datos externos. Emplea el guion `create_workspace.sh` (en `practice_assets/`) para generar el entorno del estudiante y el guion `chapter_results.sh` (en `verification/`) como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Preparar espacio de trabajo	5	Directorio creado, herramienta comprobada
2 · Paso Forward y pérdida	20–25	Activaciones y pérdida manual vs script
3 · Paso Backward y pesos	20–25	Gradientes y pesos actualizados
4 · Perturbación de η	10–15	Análisis de sensibilidad en tasa de paso
5 · Controles de saturación	10–15	Entradas nulas y gradiente en saturación
6 · Auditoría ética del modelo	10–15	Justificación de riesgos y validación
7 · Verificación y empaquetado	5	Comprobación con <code>DELIVERY_OK</code>

La ruta de este anexo suma 80–110 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo.

1 · Preparar el espacio de trabajo

Listing 1.3: Comprobacion de entorno y espacio de trabajo.

```
bash --version
chmod +x practice_assets/create_workspace.sh
./practice_assets/create_workspace.sh TC10_entrega
cd TC10_entrega
```

2 · Paso Forward y evaluación de la pérdida

Con las entradas $x_1 = 1.0$ (H3K27ac) y $x_2 = 0.5$ (ATAC-seq), y los parámetros iniciales de la red $2 \rightarrow 2 \rightarrow 1$ presentados en teoría, calcule a mano las activaciones de la capa oculta (a_{h1}, a_{h2}), la predicción final ($\hat{y}$) y la pérdida de entropía cruzada para $y = 1$.

Listing 1.4: Contrastar el paso forward.

```
bash herramienta/chapter_results.sh forward_enhancer_pass 1.0 0.5
```

El cálculo del paso forward sobre las entradas $x_1 = 1.0$ y $x_2 = 0.5$ reproduce de forma determinista la predicción $\hat{y} = 0.605264$. Se reproduce con `verification/chapter_results.sh forward_enhancer_pass 1.0 0.5`.

Listing 1.5: Evaluar la pérdida de entropía cruzada.

```
bash herramienta/chapter_results.sh bce_loss 1.0 0.605264
```

El cálculo de la pérdida de entropía cruzada binaria para $y = 1.0$ e $\hat{y} = 0.605264$ produce un valor de pérdida $\mathcal{L} = 0.502091$. Se reproduce con `verification/chapter_results.sh bce_loss 1.0 0.605264`.

3 · Paso Backward y actualización de pesos

Calcule a mano el error de salida $\delta_{\text{out}} = \hat{y} - y$, los gradientes $\frac{\partial \mathcal{L}}{\partial w_1^{(2)}}$ y $\frac{\partial \mathcal{L}}{\partial w_2^{(2)}}$, y los nuevos pesos con $\eta = 0.1$:

Listing 1.6: Calcular el paso backward y pesos actualizados.

```
bash herramienta/chapter_results.sh backward_enhancer_pass 1.0 0.5 1.0 0.1
```

El cálculo del paso backward produce los nuevos pesos actualizados de la capa de salida $w_1^{(2)} = 0.724104$ y $w_2^{(2)} = -0.380263$ con tasa de aprendizaje $\eta = 0.1$. Se reproduce con `verification/chapter_results.sh backward_enhancer_pass 1.0 0.5 1.0 0.1`.

4 · Perturbación de la tasa de aprendizaje

Incremente la tasa de aprendizaje a $\eta = 0.5$ y registre cómo cambia la magnitud de la actualización del peso $w_1^{(2)}$:

Listing 1.7: Perturbacion de la tasa de aprendizaje.

```
bash herramienta/chapter_results.sh learning_rate_perturbation 0.5
```

Perturbar la tasa de aprendizaje a $\eta = 0.5$ incrementa la actualización proporcionalmente, produciendo un nuevo peso $w_1^{(2)} = 0.820521$. Se reproduce con `verification/chapter_results.sh learning_rate_perturbation 0.5`.

5 · Controles de robustez y casos límite

Listing 1.8: Comprobar entradas nulas y saturacion.

```
bash herramienta/chapter_results.sh zero_input_control
bash herramienta/chapter_results.sh saturation_control 10.0
```

El vector de entradas nulas ($x_1 = 0, x_2 = 0$) propaga exclusivamente los sesgos a través de la red, produciendo $z_{h1} = 0.1000$ y $z_{h2} = -0.2000$. Se reproduce con `verification/chapter_results.sh zero_input_control`.

Con una preactivación saturada $z = 10.0$, la salida sigmoide es 0.999955 y el gradiente local evalúa a 0.00004540, demostrando cuantitativamente el fenómeno de desvanecimiento del gradiente en regiones saturadas. Se reproduce con `verification/chapter_results.sh saturation_control 10.0`.

6 · Auditoría de diseño metodológico y prevención de fugas de datos

Tarea. Considere un estudio clínico sintético con 1000 perfiles de expresión procedentes de 100 pacientes distribuidos en 2 hospitales. En `notas/respuestas.tsv`, identifique por qué una partición aleatoria simple a nivel de fila genera *data leakage* y describa cómo debe estructurarse la partición correcta agrupada por paciente y centro hospitalario, comparando el modelo propuesto con una regresión logística de referencia (*baseline*).

7 · Verificar la entrega

Listing 1.9: Comprobar que la entrega esta completa antes de empaquetar.

```
cd ..
../practice_assets/check_practice_delivery.sh TC10_entrega
tar -czf TC10_entrega.tar.gz TC10_entrega
sha256sum TC10_entrega.tar.gz
```

`check_practice_delivery.sh` verifica, con Bash y GNU Coreutils exclusivamente, que `notas/respuestas.tsv` existe con la cabecera esperada y al menos una fila completada, y que `herramienta/chapter_results.sh` está presente y es ejecutable.

Rúbrica de calificación ponderada

La evaluación de este anexo pondera la verificación técnica, la derivación matemática y la auditoría metodológica sobre 10 puntos (reparto 30/70 según ADR-001):

- **3,0 puntos · Entrega técnica y verificador (DELIVERY_OK):** Integridad del paquete comprimido `.tar.gz`, formato de `notas/respuestas.tsv` y superación de `check_practice_delivery.sh`.
- **2,5 puntos · Derivación de propagación directa y retropropagación:** Cálculo manual de activación oculta, salida $\hat{y}$, pérdida BCE y gradientes analíticos de pesos con precisión de 4 decimales.
- **2,5 puntos · Dinámica de optimización y casos límite:** Análisis empírico del efecto de la tasa de aprendizaje η , diagnóstico cuantitativo de saturación sigmoidea ($z = 10.0$) y propagación de sesgos con entradas nulas.
- **2,0 puntos · Auditoría ética y defensa metodológica:** Diseño riguroso de la partición clínica sin *data leakage*, comparación con un *baseline* interpretable y defensa conceptual (100–150 palabras) sobre sesgo poblacional y límites inferenciales de la IA.

Nota didáctica: La obtención de `DELIVERY_OK` acredita la ejecución de los comandos de prueba, pero no sustituye la derivación formal del cálculo multivariable ni la justificación ética del diseño experimental.

Lista de autocorrección

- Mi cálculo manual de $\hat{y}$ coincidió con el valor del script.
- Evalué la pérdida de entropía cruzada para $y = 1$.
- Calculé los gradientes de pesos y la actualización con $\eta = 0.1$.
- Registré el efecto de elevar la tasa de aprendizaje a $\eta = 0.5$.
- Comprobé la propagación de sesgos con entradas nulas.
- Verifiqué el colapso del gradiente en saturación ($z = 10.0$).
- Redacté la defensa sobre auditoría ética y sesgo demográfico.
- `check_practice_delivery.sh` finaliza con `DELIVERY_OK`.

Defensa conceptual

En 100–150 palabras, formule un protocolo de auditoría ética para un modelo de red neuronal entrenado para predecir respuesta a quimioterapia a partir de expresión génica, explicando qué precauciones deben tomarse respecto a sesgos poblacionales, fugas de datos y validación clínica independiente.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de redes neuronales en bioinformática (20251202_bioinfo_6.pptx) y se valida al 100% mediante `verification/chapter_results.sh`.